

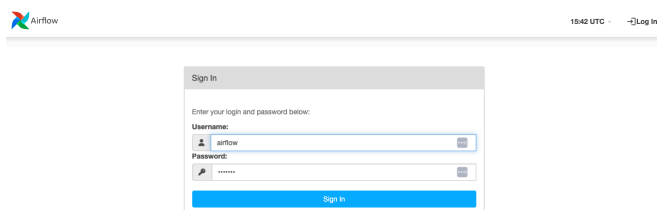
Airflow - Advanced DAGs

Recurring DAGs:

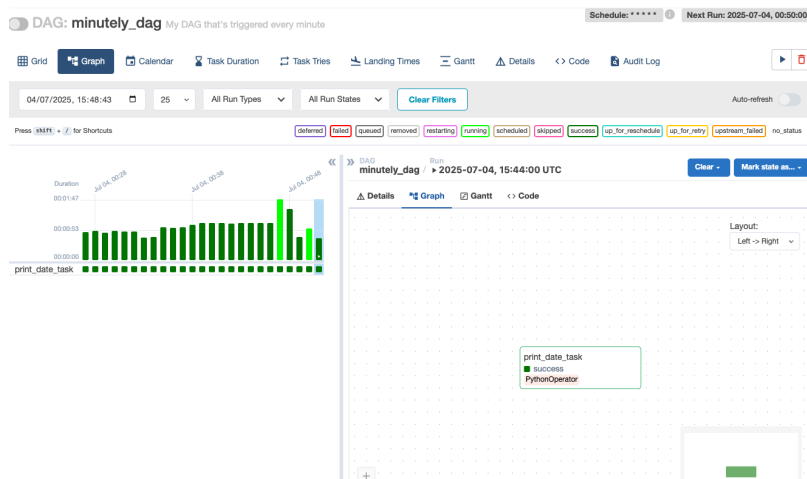
Create a DAG `minutely_dag.py` containing a task printing time and day, set DAG to run every minute

```
minutely_dag.py U X
datascientest-data-engineering > airflow > dags > minutely_dag.py
1  from airflow import DAG
2  from airflow.utils.dates import days_ago
3  from airflow.operators.python import PythonOperator
4  import datetime
5
6
7  def print_date():
8      print(datetime.datetime.now())
9
10 with DAG(
11     dag_id = 'minutely_dag',
12     description = 'My DAG that\'s triggered every minute',
13     tags = ['tutorial', 'datascientest'],
14     schedule_interval = '* * * * *',
15     default_args = {
16         'owner': 'airflow',
17         'start_date': days_ago(0, minute=1),
18     }
19 ) as my_dag:
20
21     my_task = PythonOperator(
22         task_id = 'print_date_task',
23         python_callable = print_date
24     )
```

Open URL <http://52.48.204.31:8080/> with username airflow and password airflow



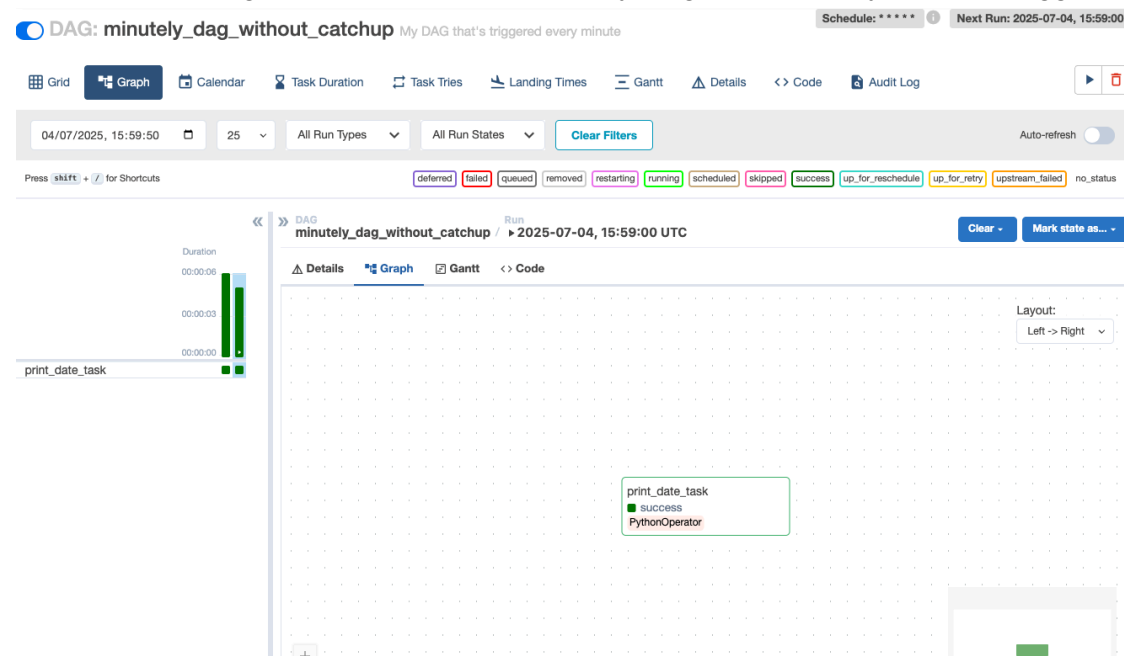
After clicking on play button to trigger, open Graph view



Press pause, then edit the minutely_dag.py file

```
minutely_dag.py U X
datascientest-data-engineering > airflow > dags > minutely_dag.py
1  from airflow import DAG
2  from airflow.utils.dates import days_ago
3  from airflow.operators.python import PythonOperator
4  import datetime
5
6
7  def print_date():
8      print(datetime.datetime.now())
9
10 with DAG(
11     dag_id = 'minutely_dag_without_catchup',
12     description = 'My DAG that\'s triggered every minute',
13     tags = ['tutorial', 'datascientest'],
14     schedule_interval = '* * * * *',
15     default_args = {
16         'owner': 'airflow',
17         'start_date': days_ago(0, minute=1),
18     },
19     catchup=False
20 ) as my_dag:
21
22     my_task = PythonOperator(
23         task_id = 'print_date_task',
24         python_callable = print_date
25     )
```

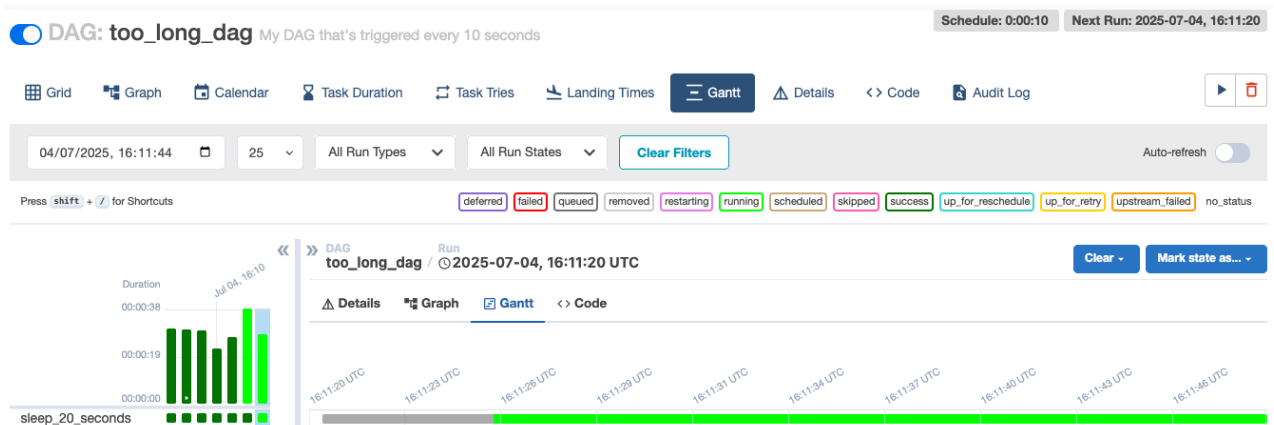
Open Airflow UI again, search for DAG minutely_dag and click play button to trigger



Create a DAG `too_long_dag.py`, this DAG takes longer to execute than expected `schedule_interval`

```
too_long_dag.py U X
datascientest-data-engineering > airflow > dags > too_long_dag.py
1  from airflow import DAG
2  from airflow.utils.dates import days_ago
3  from airflow.operators.python import PythonOperator
4  import datetime
5  import time
6
7
8  def sleep_20_seconds():
9      time.sleep(20)
10
11  with DAG(
12      dag_id = 'too_long_dag',
13      description = 'My DAG that\'s triggered every 10 seconds',
14      tags = ['tutorial', 'datascientest'],
15      schedule_interval = datetime.timedelta(seconds=10),
16      default_args = {
17          'owner': 'airflow',
18          'start_date': days_ago(0, minute=1),
19      },
20      catchup = False
21  ) as my_dag:
22
23      my_task = PythonOperator(
24          task_id = 'sleep_20_seconds',
25          python_callable = sleep_20_seconds
26      )
```

Open Airflow UI, search for DAG `too_long_dag` and click play button to trigger. Note: DAG takes at least 20 seconds to run, whereas it launches every 10 seconds.

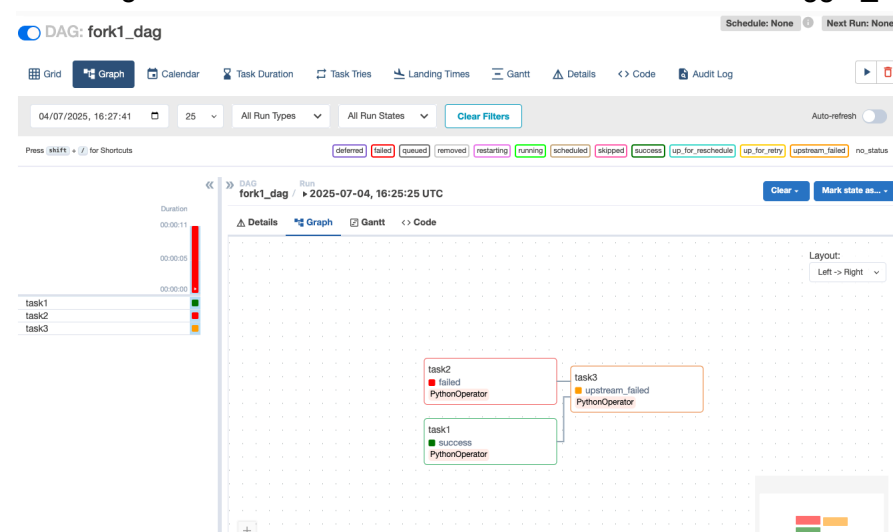


Conditional workflows:

Create DAG fork1_dag.py, a simple fork DAG to demo trigger_rule.

```
fork1_dag.py U X
datascientest-data-engineering > airflow > dags > fork1_dag.py
1 import random
2 from airflow import DAG
3 from airflow.utils.dates import days_ago
4 from airflow.operators.python import PythonOperator
5
6
7 def successful_task():
8     print('success')
9
10 def failed_task():
11     raise Exception('This task did not work!')
12
13 def random_fail_task():
14     random.seed()
15     if random.random() < .9:
16         raise Exception('This task randomly failed')
17
18 with DAG(
19     dag_id = 'fork1_dag',
20     tags = ['tutorial', 'datascientest'],
21     schedule_interval = None,
22     default_args = {
23         'owner': 'airflow',
24         'start_date': days_ago(0, minute=1)
25     },
26     catchup = False
27 ) as my_dag:
28     task1 = PythonOperator(
29         task_id = 'task1',
30         python_callable = successful_task
31     )
32
33     task2 = PythonOperator(
34         task_id = 'task2',
35         python_callable = failed_task
36     )
37
38     task3 = PythonOperator(
39         task_id = 'task3',
40         python_callable = successful_task
41     )
42
43     [task1, task2] >> task3
```

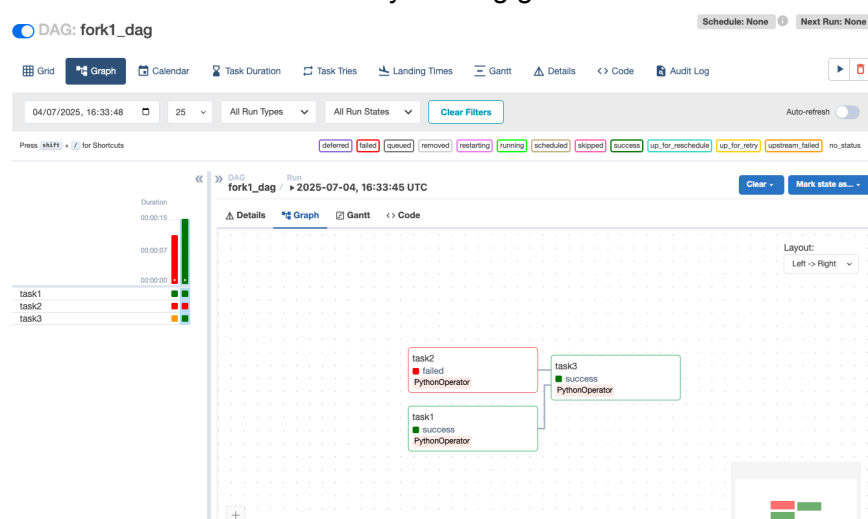
Open Airflow UI again, search for DAG fork1_dag and click play button to trigger. DAG cannot execute correctly as task2 task will inevitably fail and prevent task3 task from executing. This behaviour is due to the default value of the trigger_rule argument.



Edit file fork1_dag.py to fix with trigger_rule='all done' to change the task3 task

```
fork1_dag.py U X
datascientest-data-engineering > airflow > dags > fork1_dag.py
1  import random
2  from airflow import DAG
3  from airflow.utils.dates import days_ago
4  from airflow.operators.python import PythonOperator
5
6
7  def successful_task():
8      print('success')
9
10 def failed_task():
11     raise Exception('This task did not work!')
12
13 def random_fail_task():
14     random.seed()
15     if random.random() < .9:
16         raise Exception('This task randomly failed')
17
18 with DAG(
19     dag_id = 'fork1_dag',
20     tags = ['tutorial', 'datascientest'],
21     schedule_interval = None,
22     default_args = {
23         'owner': 'airflow',
24         'start_date': days_ago(0, minute=1)
25     },
26     catchup = False
27 ) as my_dag:
28     task1 = PythonOperator(
29         task_id = 'task1',
30         python_callable = successful_task
31     )
32
33     task2 = PythonOperator(
34         task_id = 'task2',
35         python_callable = failed_task
36     )
37
38     task3 = PythonOperator(
39         task_id = 'task3',
40         python_callable = successful_task,
41         trigger_rule = 'all_done'
42     )
43
44     [task1, task2] >> task3
```

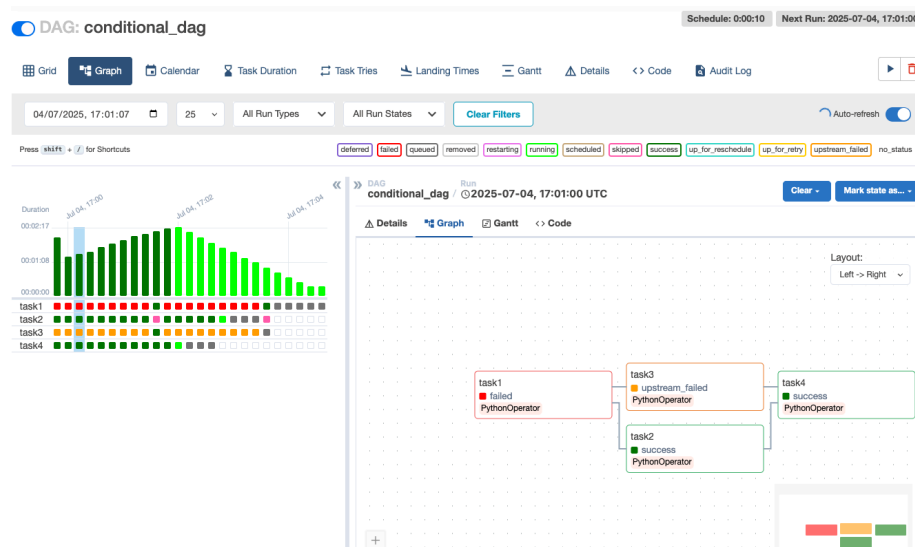
Open Airflow UI and search again for DAG fork1_dag and click play button to trigger (again). Task3 task is now successfully running green.



Create a full conditional “branching” DAG conditional_dag.py which takes a schedule_interval of 10 seconds to be able to see the different behaviours.

```
conditional_dag.py U X
datascientest-data-engineering > airflow > dags > conditional_dag.py
1  from airflow import DAG
2  from airflow.utils.dates import days_ago
3  from airflow.operators.python import PythonOperator
4  import random
5  import datetime
6
7
8  def successful_task():
9      print('success')
10
11  def failed_task():
12      raise Exception('This task did not work!')
13
14  def random_fail_task():
15      random.seed()
16      a = random.random()
17      print(a)
18      if a < .9:
19          raise Exception('This task randomly failed')
20
21  with DAG(
22      dag_id = 'conditional_dag',
23      tags = ['tutorial', 'datascientest'],
24      schedule_interval = datetime.timedelta(seconds=10),
25      default_args = {
26          'owner': 'airflow',
27          'start_date': days_ago(0, minute=1)
28      },
29      catchup = False
30  ) as my_dag:
31      task1 = PythonOperator(
32          task_id = 'task1',
33          python_callable = random_fail_task
34      )
35
36      task2 = PythonOperator(
37          task_id = 'task2',
38          python_callable = successful_task,
39          trigger_rule = 'all_failed'
40      )
41
42      task3 = PythonOperator(
43          task_id = 'task3',
44          python_callable = successful_task,
45          trigger_rule = 'all_success'
46      )
47
48      task4 = PythonOperator(
49          task_id = 'task4',
50          python_callable = successful_task,
51          trigger_rule = 'all_done'
52      )
53
54      task1 >> [task2, task3]
55      [task2, task3] >> task4
56
```

Open Airflow UI and search for DAG conditional_dag and click play button to trigger.

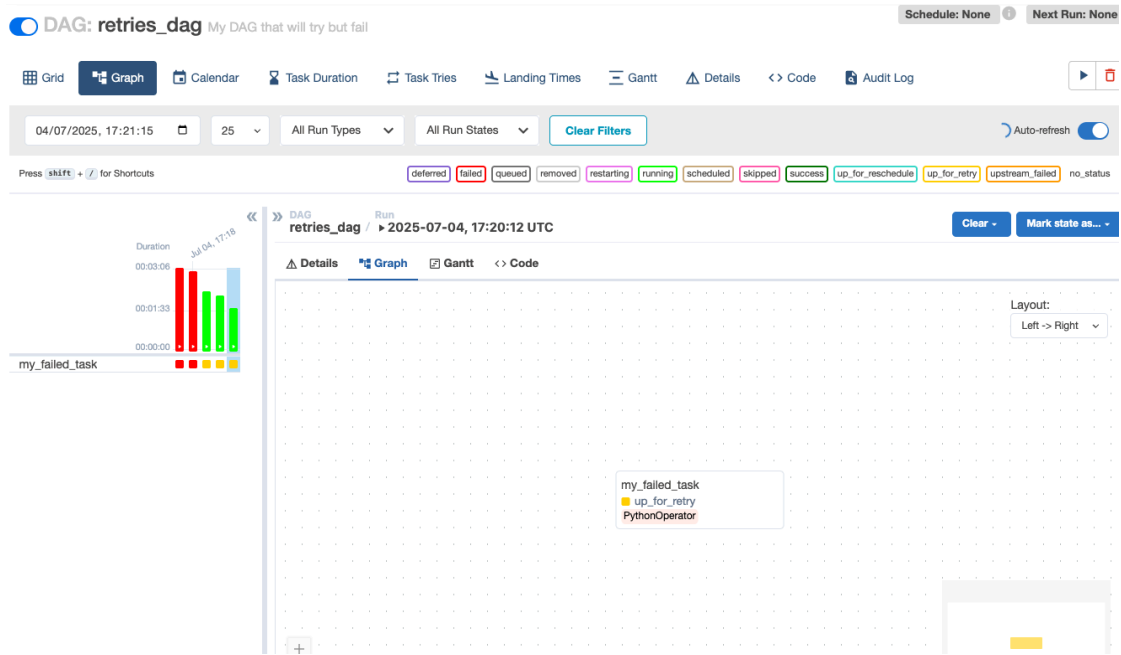


New attempts:

For the purposes of retries on failure create DAG retries_dag.py

```
retries_dag.py U X
tascientest-data-engineering > airflow > dags > retries_dag.py
1 from airflow import DAG
2 from airflow.utils.dates import days_ago
3 from airflow.operators.python import PythonOperator
4 import datetime
5
6
7 def failed_task():
8     raise Exception('This task did not work!')
9
10 with DAG(
11     dag_id = 'retries_dag',
12     description = 'My DAG that will try but fail',
13     tags = ['tutorial', 'datascientest'],
14     schedule_interval = None,
15     default_args = {
16         'owner': 'airflow',
17         'start_date': days_ago(0, minute=1),
18     },
19     catchup = False
20 ) as my_dag:
21     task1 = PythonOperator(
22         task_id = "my_failed_task",
23         python_callable = failed_task,
24         retries = 5,
25         retry_delay = datetime.timedelta(seconds=30)
26     )
```

Open Airflow UI and search for DAG retries_dag and click play button to trigger.

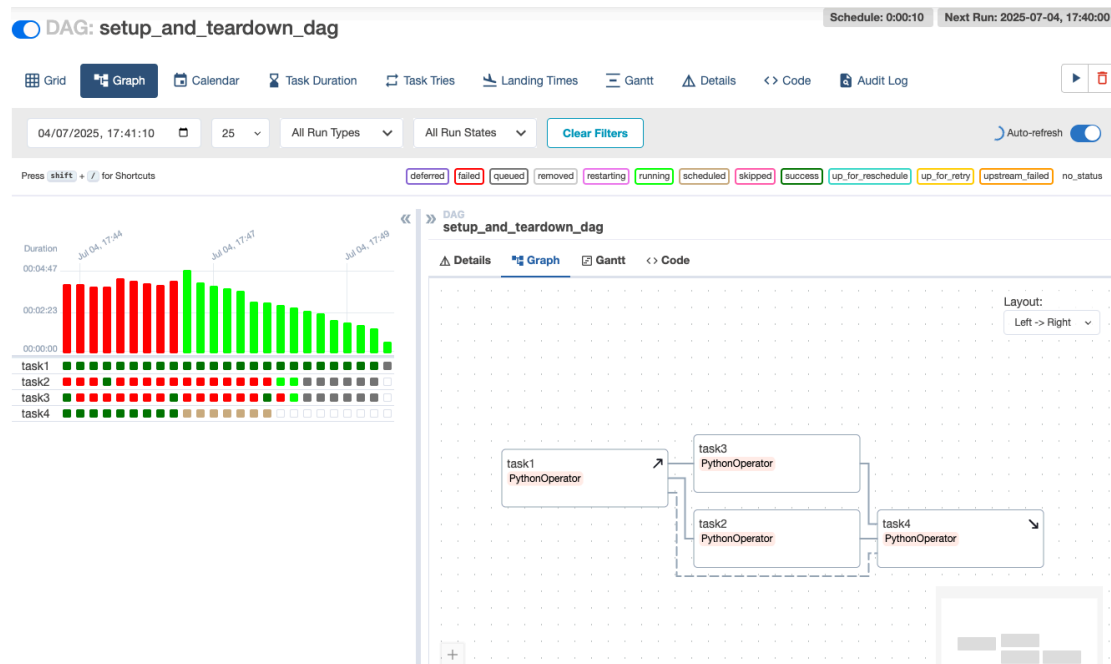


Setup and Teardown:

Create a setup_and_teardown_dag.py for setup and teardown hooks

```
setup_and_teardown_dag.py U X
datascientest-data-engineering > airflow > dags > setup_and_teardown_dag.py
1 from airflow import DAG
2 from airflow.utils.dates import days_ago
3 from airflow.operators.python import PythonOperator
4 import random
5 import datetime
6
7
8 def successful_task():
9     print('success')
10
11 def failed_task():
12     raise Exception('This task did not work!')
13
14 def random_fail_task():
15     random.seed()
16     a = random.random()
17     print(a)
18     if a < .9:
19         raise Exception('This task randomly failed')
20
21 with DAG(
22     dag_id = 'setup_and_teardown_dag',
23     tags = ['tutorial', 'datascientest'],
24     schedule_interval = datetime.timedelta(seconds=10),
25     default_args = {
26         'owner': 'airflow',
27         'start_date': days_ago(0, minute=1)
28     },
29     catchup = False
30 ) as my_dag:
31     task1 = PythonOperator(
32         task_id = 'task1',
33         python_callable = successful_task
34     )
35
36     task2 = PythonOperator(
37         task_id = 'task2',
38         python_callable = random_fail_task
39     )
40
41     task3 = PythonOperator(
42         task_id = 'task3',
43         python_callable = random_fail_task
44     )
45
46     task4 = PythonOperator(
47         task_id = 'task4',
48         python_callable = successful_task
49     )
50
51     setup = task1.as_setup()
52     teardown = task4.as_teardown()
53     setup >> [task2, task3]
54     [task2, task3] >> teardown
55     task1 >> task4
```

Open Airflow UI and search for DAG `setup_and_tearardown_dag` and click play button to trigger.



Notifications:

Create a `notifications_dag.py` DAG. Add email in `.env` file for security purposes.

```
notifications_dag.py U X
tascientest-data-engineering > airflow > dags > notifications_dag.py
1  from airflow import DAG
2  from airflow.utils.dates import days_ago
3  from airflow.operators.python import PythonOperator
4  import os
5  import datetime
6
7
8  def failed_task():
9      raise Exception("This task did not work!")
10
11 # email address in .env
12 ALERT_EMAIL = os.environ.get("ALERT_EMAIL")
13
14 with DAG(
15     dag_id = "notifications_dag",
16     description = "Retries + email notifications on failure",
17     schedule_interval = None,
18     catchup = False,
19     default_args = {
20         "owner": "airflow",
21         "start_date": days_ago(0, minute=1),
22     },
23     tags = ["tutorial", "datascientest"]
24 ) as dag:
25
26     task1 = PythonOperator(
27         task_id = "my_failed_task",
28         python_callable = failed_task,
29         retries = 5,
30         retry_delay = datetime.timedelta(seconds=30),
31
32         email_on_retry = True,
33         email_on_failure = True,
34         email = [ALERT_EMAIL]
35     )
```

Update docker-compose.yml file to include **ALERT_EMAIL: "\${ALERT_EMAIL}"**

Restart the stack

```
ubuntu@ip-172-31-46-112:~/datascientest-data-engineering/airflow$ docker-compose down
Stopping airflow_flow_1 ... done
Stopping airflow_webserver_1 ... done
Stopping airflow_scheduler_1 ... done
Stopping airflow_worker_1 ... done
Stopping airflow_postgres_1 ... done
Stopping airflow_redis_1 ... done
Removing airflow_flow_1 ... done
Removing airflow_webserver_1 ... done
Removing airflow_scheduler_1 ... done
Removing airflow_worker_1 ... done
Removing airflow_init_1 ... done
Removing airflow_postgres_1 ... done
Removing airflow_redis_1 ... done
Removing network airflow_default
ubuntu@ip-172-31-46-112:~/datascientest-data-engineering/airflow$ docker-compose up -d
Building with native build. Learn about native build in Compose here: https://docs.docker.com/go/compose-native-build/
Creating airflow_redis_1 ... done
Creating airflow_postgres_1 ... done
Creating airflow_worker_1 ... done
Creating airflow_scheduler_1 ... done
Creating airflow_init_1 ... done
Creating airflow_flow_1 ... done
Creating airflow_webserver_1 ... done
ubuntu@ip-172-31-46-112:~/datascientest-data-engineering/airflow$
```

Open Airflow UI, search for notifications_dag and trigger DAG by selecting the play button.

The screenshot displays the Apache Airflow web interface. At the top, the DAG is identified as 'notifications_dag' with the note 'Retries + email notifications on failure'. The interface includes a navigation bar with various views (Grid, Graph, Calendar, Task Duration, Task Tries, Landing Times, Gantt, Details, Code, Audit Log) and filters. The main content area shows a list of DAG runs for 'notifications_dag' with columns for 'Run ID', 'Status', 'Start Date', and 'End Date'. A specific run is selected, showing details for 'my_failed_task' with a status of 'queued'. The task instance details show a 'PythonOperator' with a duration of 00:00:05. A 'Task Instance Notes' section is also visible. The bottom part of the screenshot shows the 'Graph' view of the DAG, with a task node 'my_failed_task' highlighted in green, indicating it is running. A tooltip for the task shows its ID, status, start date, duration, and trigger rule.